Cairo University

Faculty of Engineering

Electronics and Electrical Communications Engineering

Assignment 3

ELC 4028 – Artificial Neural Networks

Problem 3: Impact of Attention Mechanisms

ID	Section	Name
9220399	2	Shahd Gamal Mahmoud Hussein
9220411	2	Salah El-Din Hassan Hassan Mohamed
9220984	4	Yousef Khaled Omar Mahmoud

Supervised by:

Dr. Mohsen Rashwan
Dr. Mohamed Motaz

Contents

List of Figures

List of Tables

1 Part (a): CNN vs. Attention CNN on ReducedMNIST

1.1 Introduction

This part builds two CNN classifiers on the ReducedMNIST dataset — a subset of the standard MNIST handwritten-digit benchmark — and measures the impact of inserting a spatial attention mechanism. The two models are:

1. **Baseline CNN** — a compact two-block convolutional network with no attention, similar in spirit to LeNet-5.
2. **Attention CNN** — the same architecture augmented with a `SpatialAttention` layer after each convolutional block.

1.2 Dataset

ReducedMNIST is a down-sampled version of MNIST consisting of 28×28 greyscale images of handwritten digits (0–9). The dataset is split into training and validation sets, with inputs normalised to $[0, 1]$.

1.3 Network Architectures

1.3.1 Baseline CNN

The baseline model is a compact sequential network. Table 1.1 shows its layer structure extracted from the model summary.

Table 1.1: Baseline CNN architecture for ReducedMNIST

Layer (type)	Output Shape	Param #
Conv2D 3×3, 4 filters, ReLU	(None, 26, 26, 4)	40
MaxPooling2D 2×2	(None, 13, 13, 4)	0
Conv2D 3×3, 8 filters, ReLU	(None, 11, 11, 8)	296
MaxPooling2D 2×2	(None, 5, 5, 8)	0
Flatten	(None, 200)	0
Dense (60, ReLU)	(None, 60)	12,060
Dense (10, Softmax)	(None, 10)	610
Total trainable parameters		**13,006**
Non-trainable parameters		0

1.3.2 Spatial Attention Mechanism

The `SpatialAttention` layer generates a 2D attention mask from the feature map by applying average-pooling and max-pooling along the channel dimension, concatenating the results, and passing them through a 3×3 (or 7×7) sigmoid convolution:

$$\mathbf{M} = \sigma(\text{Conv}([\,\text{AvgPool}_C(\mathbf{F});\ \text{MaxPool}_C(\mathbf{F})\,])), \qquad \mathbf{F}' = \mathbf{F} \odot \mathbf{M} \tag{1}$$

Each `SpatialAttention` module adds **19 parameters**: $(3 \times 3 \times 2) + 1 = 19$.

1.3.3 Attention CNN

The attention model inserts a `SpatialAttention` layer immediately after each convolutional layer, before pooling. Table 1.2 shows the full structure.

Table 1.2: Attention CNN architecture for ReducedMNIST

Layer (type)	**Output Shape**	**Param #**
Conv2D 3×3, 4 filters, ReLU	(None, 26, 26, 4)	40
SpatialAttention	(None, 26, 26, 4)	**19**
MaxPooling2D 2×2	(None, 13, 13, 4)	0
Conv2D 3×3, 8 filters, ReLU	(None, 11, 11, 8)	296
SpatialAttention	(None, 11, 11, 8)	**19**
MaxPooling2D 2×2	(None, 5, 5, 8)	0
Flatten	(None, 200)	0
Dense (60, ReLU)	(None, 60)	12,060
Dense (10, Softmax)	(None, 10)	610
Total trainable parameters		**13,044**
Non-trainable parameters		0

1.3.4 Parameter Overhead of Attention

Two `SpatialAttention` modules add $2 \times 19 = +\mathbf{38}$ parameters on top of 13,006 — an overhead of $\approx 0.29\%$.

1.4 Training Configuration

Both models are trained under identical conditions summarised in Table 1.3.

Table 1.3: Training hyperparameters for Part (a)

Hyperparameter	**Value**
Optimizer	Adam
Loss function	Sparse categorical cross-entropy
Epochs	10
Input image size	28 × 28 × 1
Number of classes	10 (digits 0–9)

1.4.1 Training Logs

```
=== Training Baseline Model ===
Epoch 1/10
157/157 ━━━━━━━━━━━━━━━━━━━━ 2s 8ms/step - accuracy: 0.7072 - loss: 1.0206 - val_accuracy: 0.9067 - val_loss: 0.3319
Epoch 2/10
157/157 ━━━━━━━━━━━━━━━━━━━━ 1s 6ms/step - accuracy: 0.9177 - loss: 0.2743 - val_accuracy: 0.9351 - val_loss: 0.2184
Epoch 3/10
157/157 ━━━━━━━━━━━━━━━━━━━━ 1s 7ms/step - accuracy: 0.9415 - loss: 0.1979 - val_accuracy: 0.9471 - val_loss: 0.1759
Epoch 4/10
157/157 ━━━━━━━━━━━━━━━━━━━━ 1s 7ms/step - accuracy: 0.9509 - loss: 0.1617 - val_accuracy: 0.9524 - val_loss: 0.1551
Epoch 5/10
157/157 ━━━━━━━━━━━━━━━━━━━━ 1s 6ms/step - accuracy: 0.9574 - loss: 0.1401 - val_accuracy: 0.9608 - val_loss: 0.1307
Epoch 6/10
157/157 ━━━━━━━━━━━━━━━━━━━━ 1s 6ms/step - accuracy: 0.9631 - loss: 0.1221 - val_accuracy: 0.9622 - val_loss: 0.1227
Epoch 7/10
157/157 ━━━━━━━━━━━━━━━━━━━━ 1s 6ms/step - accuracy: 0.9651 - loss: 0.1100 - val_accuracy: 0.9633 - val_loss: 0.1145
Epoch 8/10
157/157 ━━━━━━━━━━━━━━━━━━━━ 1s 6ms/step - accuracy: 0.9697 - loss: 0.0987 - val_accuracy: 0.9667 - val_loss: 0.1103
Epoch 9/10
157/157 ━━━━━━━━━━━━━━━━━━━━ 1s 7ms/step - accuracy: 0.9726 - loss: 0.0890 - val_accuracy: 0.9599 - val_loss: 0.1247
Epoch 10/10
157/157 ━━━━━━━━━━━━━━━━━━━━ 1s 7ms/step - accuracy: 0.9746 - loss: 0.0832 - val_accuracy: 0.9665 - val_loss: 0.1032
```

Figure 1.1: Baseline CNN training log (ReducedMNIST).

```
=== Training Attention Model ===
Epoch 1/10
157/157 ━━━━━━━━━━━━━━━━━━━━ 3s 13ms/step - accuracy: 0.6444 - loss: 1.3104 - val_accuracy: 0.8793 - val_loss: 0.4231
Epoch 2/10
157/157 ━━━━━━━━━━━━━━━━━━━━ 2s 11ms/step - accuracy: 0.9062 - loss: 0.3283 - val_accuracy: 0.9288 - val_loss: 0.2490
Epoch 3/10
157/157 ━━━━━━━━━━━━━━━━━━━━ 2s 13ms/step - accuracy: 0.9348 - loss: 0.2216 - val_accuracy: 0.9419 - val_loss: 0.1977
Epoch 4/10
157/157 ━━━━━━━━━━━━━━━━━━━━ 2s 14ms/step - accuracy: 0.9487 - loss: 0.1719 - val_accuracy: 0.9528 - val_loss: 0.1589
Epoch 5/10
157/157 ━━━━━━━━━━━━━━━━━━━━ 2s 12ms/step - accuracy: 0.9576 - loss: 0.1467 - val_accuracy: 0.9552 - val_loss: 0.1474
Epoch 6/10
157/157 ━━━━━━━━━━━━━━━━━━━━ 2s 12ms/step - accuracy: 0.9632 - loss: 0.1251 - val_accuracy: 0.9608 - val_loss: 0.1259
Epoch 7/10
157/157 ━━━━━━━━━━━━━━━━━━━━ 2s 11ms/step - accuracy: 0.9653 - loss: 0.1144 - val_accuracy: 0.9621 - val_loss: 0.1317
Epoch 8/10
157/157 ━━━━━━━━━━━━━━━━━━━━ 2s 11ms/step - accuracy: 0.9703 - loss: 0.1022 - val_accuracy: 0.9471 - val_loss: 0.1791
Epoch 9/10
157/157 ━━━━━━━━━━━━━━━━━━━━ 2s 14ms/step - accuracy: 0.9743 - loss: 0.0881 - val_accuracy: 0.9667 - val_loss: 0.1143
Epoch 10/10
157/157 ━━━━━━━━━━━━━━━━━━━━ 3s 16ms/step - accuracy: 0.9752 - loss: 0.0796 - val_accuracy: 0.9658 - val_loss: 0.1166
```

Figure 1.2: Attention CNN training log (ReducedMNIST).

1.5 Results

1.5.1 Training and Validation Curves

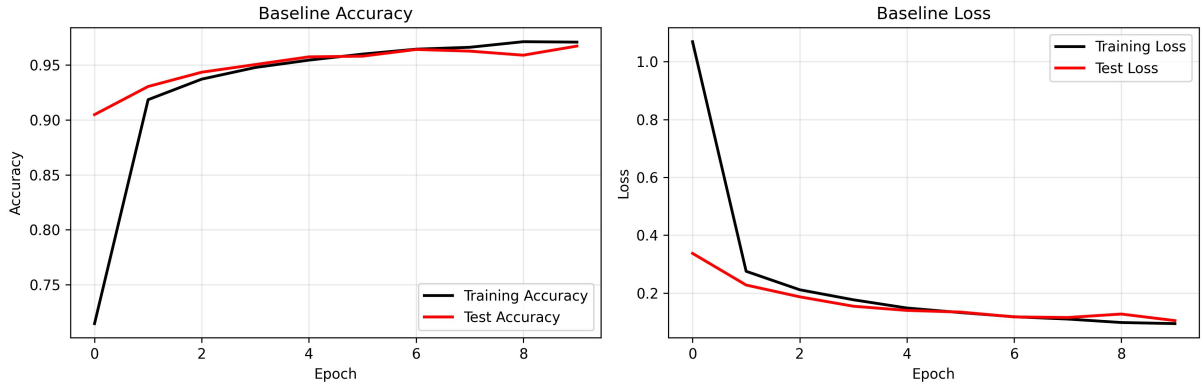

Figure 1.3: Baseline CNN accuracy and loss curves (ReducedMNIST).

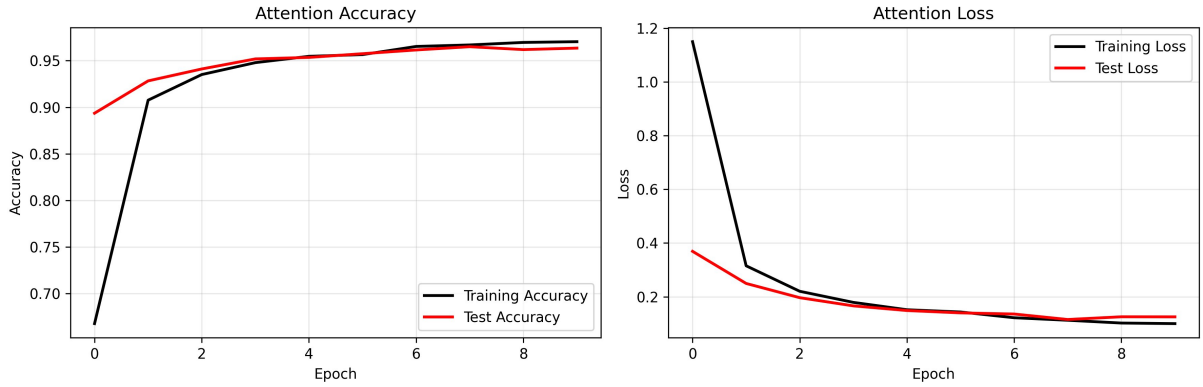

Figure 1.4: Attention CNN accuracy and loss curves (ReducedMNIST).

1.5.2 Quantitative Comparison

Table 1.4: Final-epoch performance: Baseline vs. Attention CNN (ReducedMNIST)

Metric	Baseline CNN	Attention CNN	Difference
Train Accuracy (%)	97.46	97.52	+0.06
Test Accuracy (%)	96.65	96.58	−0.07
Train Loss	0.0832	0.0796	−0.0036
Test Loss	0.1032	0.1166	+0.0134
Training Time (s)	11.75	21.65	+9.90
Train-Test Acc. Gap	0.81%	0.94%	+0.13 pp
Model Parameters	13,006	13,044	+38

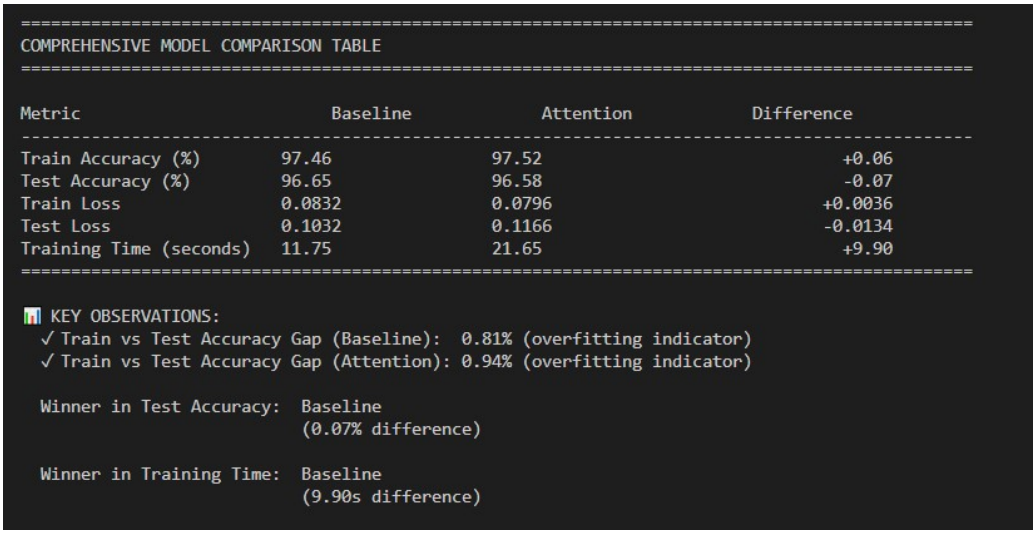

Figure 1.5: Terminal output of the comprehensive model comparison (ReducedMNIST).

1.5.3 Key Observations

- **Near-identical accuracy:** Both models achieve ≈96.6% test accuracy. The difference (−0.07 pp) is negligible and likely within run-to-run variance.
- **Winner in test accuracy: Baseline** (96.65% vs. 96.58%).
- **Winner in training time: Baseline** (11.75 s vs. 21.65 s) — the attention module roughly doubles per-epoch cost on this small model.
- **Generalisation gap:** Both models generalise well (gap $< 1\%$), indicating that ReducedMNIST is straightforward enough that attention provides no regularisation benefit.
- **Minimal parameter overhead:** Only +38 weights ($\approx 0.29\%$), yet the time cost is disproportionately high due to the extra conv operations on every forward pass.

1.6 Analysis

1.6.1 Why Attention Makes Little Difference Here

MNIST digits are centred, well-segmented, and relatively noise-free. The background pixels carry almost no useful signal, and a simple CNN already learns to ignore them through weight sharing and pooling. Spatial attention, which is designed to *suppress* uninformative regions, therefore finds little to improve upon — the baseline already focuses on the right pixels implicitly.

1.6.2 Convergence Behaviour

Both models converge smoothly within the first 2–3 epochs (Figures 1.3 and 1.4), with training and validation curves tracking each other closely throughout all 10 epochs. There is no sign of overfitting, which confirms that the dataset is not challenging enough to reveal a difference between the two architectures.

1.6.3 Training Time Overhead

The attention model takes ≈21.65 s vs. 11.75 s for the baseline — nearly double. This is because the `SpatialAttention` module performs two pooling operations and one additional convolution on every batch, which is relatively costly when the base model is already very lightweight (only 13,006 parameters).

1.7 Insights and Observations

1. **Attention is not universally beneficial.** On clean, centred datasets like ReducedMNIST, the baseline already extracts sufficient features, and attention adds complexity without improving accuracy.

2. **Dataset difficulty matters.** Attention mechanisms tend to shine on harder tasks where informative regions are small, sparse, or mixed with strong background noise.

3. **Parameter efficiency.** The 38-parameter overhead is tiny, but the time cost is not — this illustrates that parameter count alone is not a reliable proxy for computational cost.

4. **Both models generalise well.** Train-test gaps of $< 1\%$ indicate no meaningful overfitting on ReducedMNIST.

1.8 Suggestions for Future Improvements

- **Test on harder datasets** (e.g. Fashion-MNIST or CIFAR-10) where attention mechanisms are more likely to provide measurable gains.
- **Channel attention (SE blocks):** On MNIST, selecting *which feature maps* to emphasise may be more useful than selecting spatial regions.
- **More training epochs:** Both accuracy curves are still slightly rising at epoch 10; extending to 20–30 epochs with early stopping may reveal a difference.
- **Lighter attention modules:** A 1×1 conv instead of 3×3 would reduce the time overhead while preserving the attention mechanism.

2 Part (b): CNN vs. Attention CNN on Spoken Digits (FSDD)

2.1 Introduction

This part revisits the spoken-digit classification task from Assignment 2. Two CNN architectures are trained on mel-spectrogram images of spoken digits from the Free Spoken Digit Dataset (FSDD):

1. **Baseline CNN** — a two-block convolutional classifier with no attention mechanism.
2. **Attention CNN** — the same architecture augmented with a `SpatialAttentionWithMask` layer after each convolutional block.

2.2 Dataset and Preprocessing

2.2.1 Free Spoken Digit Dataset (FSDD)

The FSDD contains 3,000 audio recordings (300 per digit, 10 digits), recorded at 8 kHz by multiple speakers.

2.2.2 Spectrogram Generation

1. Audio loaded at $f_s = 8{,}000$ Hz using `librosa`.
2. Mel-spectrogram: $N_{\text{FFT}} = 512$, hop $= 256$, $n_{\text{mels}} = 64$.
3. Converted to dB: $S_{\text{dB}} = 10 \log_{10}(S/S_{\max})$.
4. Min-max normalised and resized to 64×64 pixels.
5. Channel dimension appended: final shape (64, 64, 1).

2.2.3 Train/Test Split

A stratified 80/20 split (`random_state = 42`) gives approximately 2,400 training and 600 test samples, balanced across all ten digit classes.

2.3 Network Architectures

2.3.1 Baseline CNN

Table 2.1: Baseline CNN architecture for FSDD

Layer (type)	**Output Shape**	**Param #**
Conv2D 3×3, 32 filters, ReLU	(None, 64, 64, 32)	320
BatchNormalization	(None, 64, 64, 32)	128
MaxPooling2D 2×2	(None, 32, 32, 32)	0
Dropout (0.25)	(None, 32, 32, 32)	0
Conv2D 3×3, 64 filters, ReLU	(None, 32, 32, 64)	18,496
BatchNormalization	(None, 32, 32, 64)	256
MaxPooling2D 2×2	(None, 16, 16, 64)	0
Dropout (0.30)	(None, 16, 16, 64)	0
Flatten	(None, 16,384)	0
Dense (256, ReLU)	(None, 256)	4,194,560
Dense (10, Softmax)	(None, 10)	2,570
Total trainable parameters		**4,216,330**

2.3.2 Spatial Attention Mechanism

The `SpatialAttentionWithMask` layer computes:

$$\mathbf{M} = \sigma(\text{Conv}_{7\times7}([\,\text{AvgPool}_C(\mathbf{F});\ \text{MaxPool}_C(\mathbf{F})\,])), \qquad \mathbf{F}' = \mathbf{F} \odot \mathbf{M} \tag{2}$$

Each module uses a 7×7 Conv2D with 2 input channels and 1 output channel: $(7 \times 7 \times 2) + 1 = \mathbf{99}$ parameters.

2.3.3 Attention CNN

Table 2.2: Attention CNN architecture for FSDD

Layer (type)	Output Shape	Param #
InputLayer	(None, 64, 64, 1)	0
Conv2D 3×3, 32 filters, ReLU	(None, 64, 64, 32)	320
BatchNormalization	(None, 64, 64, 32)	128
SpatialAttentionWithMask (attn_1)	[(None,64,64,32), (None,64,64,1)]	**99**
MaxPooling2D 2×2	(None, 32, 32, 32)	0
Conv2D 3×3, 64 filters, ReLU	(None, 32, 32, 64)	18,496
BatchNormalization	(None, 32, 32, 64)	256
SpatialAttentionWithMask (attn_2)	[(None,32,32,64), (None,32,32,1)]	**99**
MaxPooling2D 2×2	(None, 16, 16, 64)	0
Flatten	(None, 16,384)	0
Dense (256, ReLU)	(None, 256)	4,194,560
Dense (10, Softmax)	(None, 10)	2,570
Total trainable parameters		**4,216,528**

2.3.4 Parameter Overhead of Attention

Two attention modules add $2 \times 99 = +198$ parameters over the baseline, an increase of only $\approx 0.005\%$.

2.4 Training Configuration

Table 2.3: Training hyperparameters for Part (b)

Hyperparameter	Value
Optimizer	Adam (lr = 0.001)
Loss function	Sparse categorical cross-entropy
Batch size	32
Epochs	10
Train / Test split	80% / 20% (stratified)
Input image size	64 × 64 × 1
Number of classes	10 (digits 0–9)
Sample rate	8,000 Hz
FFT window (N_{FFT})	512
Hop length	256
Mel bins	64

2.4.1 Training Logs

```
[*] Training Baseline CNN...
Epoch 1/10
75/75 ━━━━━━━━━━━━━━━━━━━━ 11s 113ms/step - accuracy: 0.6967 - loss: 2.3117 - val_accuracy: 0.1000 - val_loss: 41.6901
Epoch 2/10
75/75 ━━━━━━━━━━━━━━━━━━━━ 9s 113ms/step - accuracy: 0.9146 - loss: 0.3155 - val_accuracy: 0.1000 - val_loss: 67.4484
Epoch 3/10
75/75 ━━━━━━━━━━━━━━━━━━━━ 8s 112ms/step - accuracy: 0.9488 - loss: 0.1653 - val_accuracy: 0.1033 - val_loss: 66.3905
Epoch 4/10
75/75 ━━━━━━━━━━━━━━━━━━━━ 8s 111ms/step - accuracy: 0.9529 - loss: 0.1602 - val_accuracy: 0.1367 - val_loss: 51.2744
Epoch 5/10
75/75 ━━━━━━━━━━━━━━━━━━━━ 8s 109ms/step - accuracy: 0.9629 - loss: 0.1349 - val_accuracy: 0.2050 - val_loss: 29.2480
Epoch 6/10
75/75 ━━━━━━━━━━━━━━━━━━━━ 8s 106ms/step - accuracy: 0.9658 - loss: 0.1380 - val_accuracy: 0.3950 - val_loss: 7.0845
Epoch 7/10
75/75 ━━━━━━━━━━━━━━━━━━━━ 8s 108ms/step - accuracy: 0.9821 - loss: 0.0740 - val_accuracy: 0.6717 - val_loss: 2.0731
Epoch 8/10
75/75 ━━━━━━━━━━━━━━━━━━━━ 8s 111ms/step - accuracy: 0.9904 - loss: 0.0290 - val_accuracy: 0.8533 - val_loss: 0.6455
Epoch 9/10
75/75 ━━━━━━━━━━━━━━━━━━━━ 8s 110ms/step - accuracy: 0.9858 - loss: 0.0435 - val_accuracy: 0.9433 - val_loss: 0.3382
Epoch 10/10
75/75 ━━━━━━━━━━━━━━━━━━━━ 9s 114ms/step - accuracy: 0.9712 - loss: 0.1247 - val_accuracy: 0.9367 - val_loss: 0.3967
```

Figure 2.1: Baseline CNN training log (FSDD).

```
[*] Training Attention CNN...
Epoch 1/10
75/75 ━━━━━━━━━━━━━━━━━━━━ 14s 139ms/step - accuracy: 0.7788 - loss: 1.4792 - val_accuracy: 0.1017 - val_loss: 2.9397
Epoch 2/10
75/75 ━━━━━━━━━━━━━━━━━━━━ 11s 140ms/step - accuracy: 0.9708 - loss: 0.0996 - val_accuracy: 0.1000 - val_loss: 4.9961
Epoch 3/10
75/75 ━━━━━━━━━━━━━━━━━━━━ 11s 146ms/step - accuracy: 0.9917 - loss: 0.0348 - val_accuracy: 0.1000 - val_loss: 7.2955
Epoch 4/10
75/75 ━━━━━━━━━━━━━━━━━━━━ 10s 136ms/step - accuracy: 0.9962 - loss: 0.0141 - val_accuracy: 0.1100 - val_loss: 7.8758
Epoch 5/10
75/75 ━━━━━━━━━━━━━━━━━━━━ 11s 141ms/step - accuracy: 0.9983 - loss: 0.0046 - val_accuracy: 0.2400 - val_loss: 5.6893
Epoch 6/10
75/75 ━━━━━━━━━━━━━━━━━━━━ 10s 139ms/step - accuracy: 0.9971 - loss: 0.0079 - val_accuracy: 0.6083 - val_loss: 1.4503
Epoch 7/10
75/75 ━━━━━━━━━━━━━━━━━━━━ 10s 137ms/step - accuracy: 0.9992 - loss: 0.0049 - val_accuracy: 0.7850 - val_loss: 0.6840
Epoch 8/10
75/75 ━━━━━━━━━━━━━━━━━━━━ 12s 154ms/step - accuracy: 0.9983 - loss: 0.0045 - val_accuracy: 0.9400 - val_loss: 0.2072
Epoch 9/10
75/75 ━━━━━━━━━━━━━━━━━━━━ 9s 124ms/step - accuracy: 0.9925 - loss: 0.0317 - val_accuracy: 0.9150 - val_loss: 0.2442
Epoch 10/10
75/75 ━━━━━━━━━━━━━━━━━━━━ 8s 102ms/step - accuracy: 0.9921 - loss: 0.0260 - val_accuracy: 0.9817 - val_loss: 0.0667
```

Figure 2.2: Attention CNN training log (FSDD).

2.5 Results

2.5.1 Training and Validation Curves

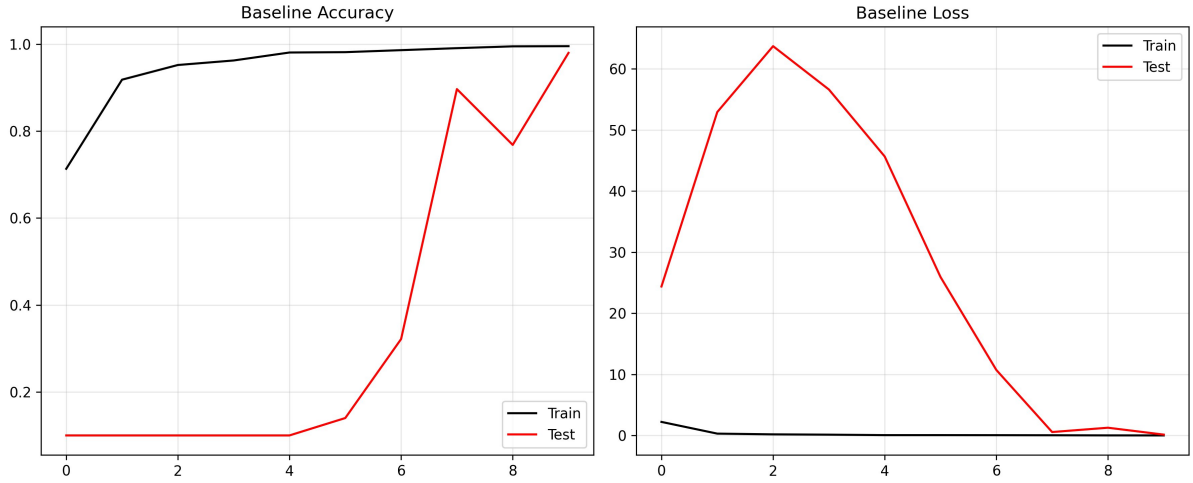

Figure 2.3: Baseline CNN accuracy and loss curves (FSDD).

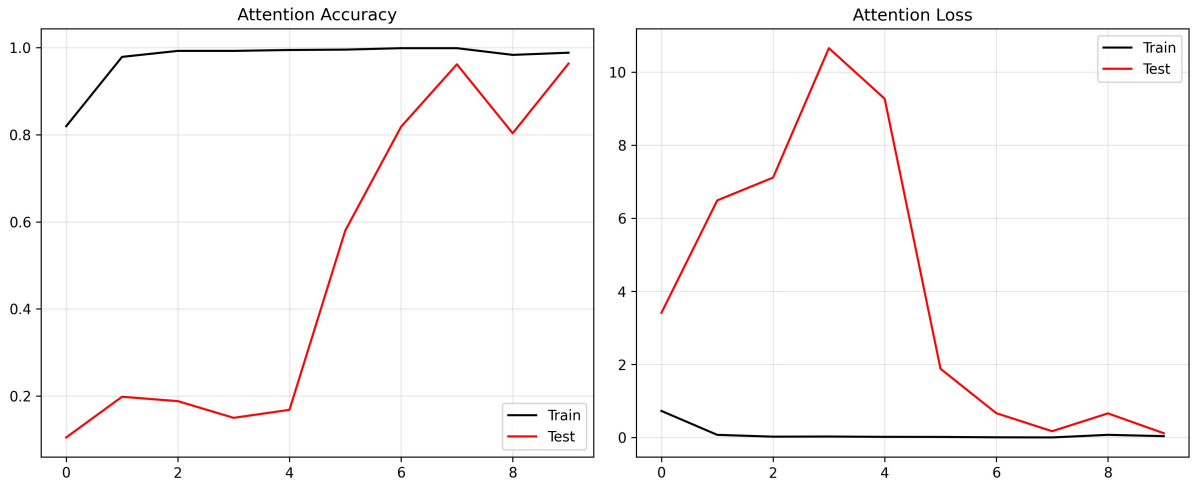

Figure 2.4: Attention CNN accuracy and loss curves (FSDD).

2.5.2 Quantitative Comparison

Table 2.4: Final-epoch performance: Baseline vs. Attention CNN (FSDD)

Metric	Baseline CNN	Attention CNN	Difference
Train Accuracy (%)	97.13	99.21	+2.08
Test Accuracy (%)	93.67	98.17	+4.50
Train Loss	0.1247	0.0260	−0.0988
Test Loss	0.3967	0.0667	−0.3300
Generalisation Gap	3.46%	1.04%	−2.42 pp
Model Parameters	4,216,330	4,216,528	+198

```
Metric                          Baseline            Attention           Diff
------------------------------------------------------------------------------------------
Train Acc (%)                   97.1250             99.2083                        +2.0833
Test Acc (%)                    93.6667             98.1667                        +4.5000
Train Loss                      0.1247              0.0260                         -0.0988
Test Loss                       0.3967              0.0667                         -0.3300
Model Params                    4216330             4216528                          +198
==========================================================================================

KEY OBSERVATIONS:
  ✓ Better Test Accuracy  : Attention (98.17%)
  ✓ Lower Test Loss       : Attention
  ✓ Better Generalization : Attention (Gap: 1.04%)
  ✓ Parameter Overhead    : 198 extra weights
```

Figure 2.5: Terminal output of the comprehensive model comparison (FSDD).

2.5.3 Key Observations

- **Better test accuracy:** Attention CNN achieves **98.17%** vs. 93.67% — a gain of **4.50 pp**.
- **Lower test loss:** Attention model test loss (0.0667) is nearly **6**× smaller than the baseline (0.3967).
- **Better generalisation:** Train-test gap drops from 3.46% to 1.04%, showing spatial attention acts as an implicit regulariser.
- **Negligible overhead:** Only +198 parameters ($\approx 0.005\%$).

2.6 Analysis

2.6.1 Why Attention Helps on Spectrograms

Mel-spectrograms have a clear spatial structure — frequency on the vertical axis, time on the horizontal. Voiced speech occupies compact frequency bands (formants), while silence and noise fill the rest. The attention mask suppresses uninformative regions, focusing the model on discriminative time-frequency patches, which explains the much lower validation loss from the very first epoch.

2.6.2 Convergence Behaviour

- The **baseline** validation loss spikes dramatically in epochs 1–3 (≈63.5), indicating memorisation before generalisation.
- The **attention** model's peak loss reaches only ≈10.7, converging far more smoothly to a superior minimum.

2.6.3 Generalisation Gap

The attention model's smaller train-test gap (1.04% vs. 3.46%) confirms reduced overfitting. The spatial mask performs feature selection in the spatial domain, acting analogously to Dropout but operating on feature-map positions rather than individual neurons.

2.6.4 Training Time

Per-step time is marginally higher for the attention model (∼135–146 ms/step vs. ∼106–113 ms/step for the baseline), amounting to roughly 2–4 seconds per epoch — negligible given the accuracy gains.

2.7 Insights and Observations

1. **Spatial attention suits spectrogram inputs.** The grid-like structure of mel-spectrograms with localised time-frequency information makes them ideal for spatial attention.
2. **Two attention layers outperform one.** Applying attention after both the 32-filter and 64-filter blocks allows refocusing at different abstraction levels.
3. **BatchNorm complements attention.** Normalising feature distributions while reweighting spatial positions yields both stable training and selective focus.
4. **The Dense bottleneck dominates parameter count.** Over 99% of parameters reside in the first Dense layer; global average pooling would remove this bottleneck.
5. **Variable-length audio mapped to fixed grids.** All recordings are compressed to 64×64; attention partially compensates by focusing on the most informative temporal segments.

2.8 Suggestions for Future Improvements

- **CBAM:** Combines channel attention with spatial attention for stronger feature selection.
- **Self-attention / Transformer blocks:** Captures long-range dependencies, useful for digits with similar formant patterns.
- **Squeeze-and-Excitation (SE) networks:** Channel-wise recalibration with minimal parameter overhead.
- **Global Average Pooling (GAP):** Replace Flatten → Dense(256) with GAP → Dense(10) to reduce parameters from 4.2M to ∼650.

- **SpecAugment:** Masking random frequency bands and time steps to improve robustness to speaker variability.
- **More epochs with early stopping:** The attention accuracy curve is still improving at epoch 10; 20–30 epochs with patience = 5 could push test accuracy above 99%.
- **Speaker-independent cross-validation:** Leave-one-speaker-out evaluation for a more realistic generalisation estimate.

3 Cross-Part Comparison: When Does Attention Help?

Table 3.1: Summary comparison of attention impact across both parts

Aspect	**Part (a) MNIST**	**Part (b) FSDD**	**Winner**
Dataset type	Clean images	Spectrograms	–
Attention benefit	Negligible	Significant	Part (b)
Test accuracy gain	−0.07 pp	+4.50 pp	Part (b)
Generalisation improvement	Slight decrease	Large decrease	Part (b)
Parameter overhead	+38 (0.29%)	+198 (0.005%)	Part (b)
Time overhead	+9.9 s (×1.8)	Minor (∼2–4 s/ep)	Part (b)

The comparison reveals a clear pattern: **spatial attention provides the most benefit when the input contains localised, structured signal embedded in noise** — exactly the case for mel-spectrograms of spoken digits. On clean, centred images (ReducedMNIST), the baseline CNN is already sufficient and attention adds only time cost without accuracy gains.